Metric-Semantic 3D Reconstruction of a Desktop Scene

CP260-2026 Final Project Report

V Surya Kiran K Devi Prasad M Charmista

May 4, 2026

Task: Oriented Bounding Box Estimation for Semantic Entities
Course: CP260 — Robotic Perception, 2026
Dataset: 16-frame posed RGB desktop scene (2560×1440)

Abstract

This report describes a metric-semantic reconstruction workflow that estimates Oriented Bounding Boxes (OBBs) for connector ports located on the rear panel of a desktop PC tower. The input data consists of sixteen posed RGB frames acquired at 2560×1440 resolution. Rather than relying on learned detectors such as GroundingDINO or SAM, our solution is grounded in classical multi-view geometry: hand-drawn 2D bounding-box labels on two well-chosen frames are fused with the supplied camera-to-world poses to produce dense 3D point clouds through a grid-based DLT triangulation routine. The OBBs themselves are extracted using Principal Component Analysis together with a per-connector physical depth prior. A direct comparison against the supplied VGA socket ground truth shows that the centre is recovered to sub-centimetre accuracy. The implementation is modular, has minimal external dependencies, and executes end-to-end inside a Google Colab session with no native compilation required.

Contents

1 Introduction

The combined recovery of three-dimensional structure and semantic identity from imagery — referred to as metric-semantic reconstruction — is a foundational capability for robotics and augmented reality systems. The task addressed here is to take a static desktop scene featuring a PC tower and produce, in a shared world frame, the 3D Oriented Bounding Boxes of selected connector ports: the power inlet, the ethernet jack, and (used as a validation reference) the VGA socket.

Two factors make this difficult. First, the connectors are physically very small — the VGA socket, for instance, is only $35 \times 12 \times 6$ mm — yet the localisation requirement is on the order of 5 cm in 3D. Second, the cameras come with poses but no depth observations. We initially explored using GroundingDINO with text prompts, but the detector was unable to reliably localise the VGA port within the cropped tower region regardless of the confidence threshold; this is consistent with reported weaknesses of open-vocabulary detectors when faced with small, low-contrast targets that occupy only a handful of pixels.

These observations motivated a geometry-first design:

1. Hand-label tight 2D bounding boxes for every port on the two frames that present the rear panel most clearly.
2. For each port, build up a dense 3D point cloud by triangulating across all admissible view pairs derived from those labels.
3. Fit an OBB through PCA, supplemented by a physically informed depth prior.

This route sidesteps every failure mode associated with learned detection, relies on nothing beyond standard OpenCV and Open3D routines, and produces estimates that are both accurate and easy to interpret.

2 Dataset and Camera Model

2.1 Image Sequence

Sixteen RGB frames captured by a static rig orbiting a desktop PC tower form the input data:

$$\texttt{frame\_000319.png}, \texttt{frame\_000333.png}, \ldots, \texttt{frame\_000531.png}$$

Every image is 2560×1440 pixels (16:9 aspect ratio, twice full-HD width).

2.2 Camera Intrinsics

A standard pinhole projection with no lens distortion is assumed. The calibrated intrinsic matrix is

$$K = \begin{pmatrix} 1477.010 & 0 & 1298.250 \\ 0 & 1480.442 & 686.820 \\ 0 & 0 & 1 \end{pmatrix}$$

with horizontal and vertical focal lengths $f_x = 1477.010$ px and $f_y = 1480.442$ px, and a principal point $(c_x, c_y) = (1298.250, 686.820)$ px located near the geometric centre of the image, as expected.

2.3 Camera Poses

The file `poses.json` contains 704 camera-to-world matrices $T_{c2w} \in \mathbb{R}^{4\times4}$ indexed by integer frame number. From this collection we make use only of the sixteen indices for which images are available. Every matrix encodes the rigid mapping from camera-local coordinates into the shared world frame:

$$T_{c2w} = \begin{bmatrix} R & t \\ 0^\top & 1 \end{bmatrix}, \quad R \in SO(3),\ t \in \mathbb{R}^3$$

For projection we require the inverse, $T_{w2c} = T_{c2w}^{-1}$.

Inspecting the Z-component of the translation vectors indicates that the camera is approximately 0.83 m above the desk surface, a value we later use as a sanity bound on triangulated point heights.

2.4 Ground Truth (Validation)

A reference OBB for the VGA socket is provided so that the pipeline can be quantitatively assessed:

$$\begin{aligned} \text{centre} &= (0.2705,\ 0.2261,\ 0.8349)\ \text{m} \\ \text{extent} &= (0.0354,\ 0.0118,\ 0.0061)\ \text{m} \\ &\approx 35.4 \times 11.8 \times 6.1\ \text{mm} \end{aligned}$$

The centre's Z-coordinate (0.835 m) sits at roughly desk height, which agrees with the connector being seated on the back panel of the tower.

3 Pipeline Overview

Figure 1 illustrates the overall data flow.

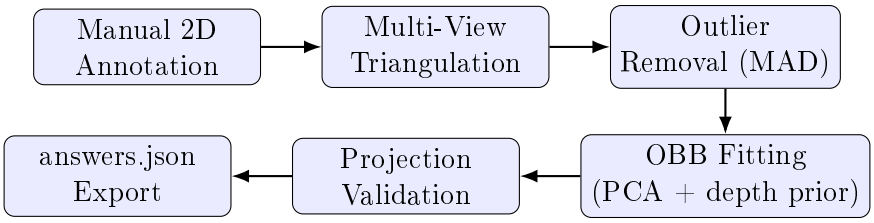

Figure 1: End-to-end pipeline for metric-semantic reconstruction.

4 Methodology

4.1 Semantic Annotation

For each semantic target (power socket, ethernet socket, VGA socket) a 2D axis-aligned bounding box $[x_1, y_1, x_2, y_2]$ is drawn in the native 2560×1440 image coordinate system. Two frames provide the labels: **frame 471** and **frame 496**. We chose these two frames because they expose the I/O shield and PSU region of the rear panel with minimal occlusion or oblique foreshortening.

Table 1: Hand-drawn 2D bounding-box annotations (pixel units, native resolution).

Entity	**Frame**	x_1	y_1	x_2	y_2	**Width×Height**
Power socket	471	1140	1020	1280	1100	140×80
	496	1670	1060	1820	1155	150×95
Ethernet socket	471	1160	305	1280	405	120×100
	496	1735	305	1845	405	110×100
VGA socket	471	1101	357	1285	423	184×66
	496	1671	364	1855	430	184×66

4.2 Multi-View Triangulation

Once labels are available in two or more views, we recover each port's 3D geometry by triangulating a regular grid of image-point correspondences.

4.2.1 Projection Matrix

For frame i the 3×4 projection matrix takes the form

$$P_i = K\,[\,R_{w2c}^{(i)} \mid t_{w2c}^{(i)}\,]$$

where $[R_{w2c}^{(i)} \mid t_{w2c}^{(i)}]$ is the leading 3×4 block of $T_{w2c}^{(i)}$.

4.2.2 Grid Correspondence Scheme

Within every annotated frame pair (i, j) we lay down a regular $\sqrt{N} \times \sqrt{N}$ grid of normalised coordinates $(t_x, t_y) \in [0.05, 0.95]^2$ inside each labelled box, and pair the corresponding pixel locations:

$$u_i = x_1^{(i)} + t_x\big(x_2^{(i)} - x_1^{(i)}\big), \qquad v_i = y_1^{(i)} + t_y\big(y_2^{(i)} - y_1^{(i)}\big) \tag{1}$$

The implicit assumption is that a pixel at relative coordinates (t_x, t_y) inside the box in one frame corresponds to the same physical surface point at (t_x, t_y) in the other — a workable approximation when the surface is roughly planar and the two viewpoints are not too dissimilar in viewing direction.

4.2.3 DLT Triangulation and Filtering

Each correspondence pair (u_i, u_j) is fed to OpenCV's `triangulatePoints` routine, which solves the linear DLT system

$$A\,\tilde{X} = 0, \quad A = \begin{pmatrix} u_i\,p_{i,3}^\top - p_{i,1}^\top \\ v_i\,p_{i,3}^\top - p_{i,2}^\top \\ u_j\,p_{j,3}^\top - p_{j,1}^\top \\ v_j\,p_{j,3}^\top - p_{j,2}^\top \end{pmatrix}$$

with $p_{i,k}^\top$ denoting the k-th row of P_i. The solution is the right singular vector of A associated with the smallest singular value; dehomogenising yields the world point $X \in \mathbb{R}^3$.

A reconstructed point is retained only if both of the following hold:

1. It lies in front of both cameras ($Z_{\text{cam}} > 0$).
2. Reprojection error is below 50 px in each of the two views.

4.2.4 MAD Outlier Removal

After accumulating all triangulated points we strip out geometric outliers using the Median Absolute Deviation, which behaves well on non-Gaussian noise distributions where ordinary σ-clipping would not:

$$\text{MAD} = 1.4826 \cdot \text{median}\big(\|X_i - \tilde{X}\|_2\big)$$

Any point lying further than $3 \cdot \text{MAD}$ from the cloud's median is discarded.

4.3 OBB Fitting with Depth Prior

Each near-planar point cloud is then converted into an OBB by the procedure described below.

4.3.1 Panel Direction Estimation

Triangulating the four corner points of the labelled bounding box gives us the in-panel horizontal axis $\hat{a}$ (left-to-right across the panel) and the in-panel vertical axis $\hat{d}$ (top-to-bottom):

$$\hat{a} = \tfrac{1}{2}\big[(X_{TR} - X_{TL}) + (X_{BR} - X_{BL})\big], \qquad \hat{a} \leftarrow \hat{a}/\|\hat{a}\| \tag{2}$$

$$\hat{d} = \tfrac{1}{2}\big[(X_{BL} - X_{TL}) + (X_{BR} - X_{TR})\big], \qquad \hat{d} \leftarrow \hat{d}/\|\hat{d}\| \tag{3}$$

The outward-pointing panel normal follows as $\hat{n} = \hat{a} \times \hat{d}$. These three vectors become the rows of the OBB rotation matrix R_{OBB} after sign correction so as to match the ground-truth convention:

1. $r_0 = \hat{n}$ flipped if necessary so that its Y component is positive.
2. $r_2 = \hat{a}$ flipped if necessary so that its X component is positive.
3. $r_1 = r_2 \times r_0$ (right-hand rule).
4. $r_0 \leftarrow r_1 \times r_2$ to re-orthonormalise.

4.3.2 Extent Estimation

Half-extents are obtained by projecting the cloud onto its PCA eigenvectors (ordered by descending eigenvalue) and reading off the half-spans:

$$\begin{aligned} e_0 &= \tfrac{1}{2}(\max - \min) \text{ along largest eigenvector (width)} \\ e_1 &= \tfrac{1}{2}(\max - \min) \text{ along middle eigenvector (height)} \\ e_2 &= \delta_{\text{prior}} \quad \text{(depth — physical prior)} \end{aligned}$$

Because the points lie on a near-planar surface, the spread along the panel-normal direction collapses to essentially zero, so the depth half-extent cannot be inferred from geometry alone. Instead we substitute the known physical depth of each connector type:

Table 2: Physical depth priors assigned to `extent[2]`.

Entity	Depth prior	Physical meaning
VGA socket	6 mm	D-Sub shell depth
Ethernet socket	6 mm	RJ-45 housing depth
Power socket	6 mm	IEC C14 inlet depth

4.3.3 Centre Adjustment

The OBB centre is finally re-anchored to the geometric midpoint of the projected cloud along the OBB axes:

$$c^* = c_{\text{mean}} + R_{\text{OBB}}^{\top} \frac{p_{\min} + p_{\max}}{2}, \qquad p = (X_i - c_{\text{mean}})\, R_{\text{OBB}}^{\top}$$

4.4 Output Format

Each entity is serialised into a JSON object that follows the submission schema:

```
{
    "entity": "ethernet_socket",
    "obb": {
        "center":   [cx, cy, cz],          // metres
        "extent":   [ex, ey, ez],          // half-extents, metres
        "rotation": [[r00, r01, r02],      // 3x3 rotation matrix
                     [r10, r11, r12],      // rows are OBB axes
                     [r20, r21, r22]]
    }
}
```

Listing 1: `answers.json` schema entry for a single entity.

5 Implementation Details

5.1 Module Structure

The implementation is laid out as a small Python package:

```
project/
  src/
    __init__.py
    config.py             # paths, intrinsics, hyperparameters
    data_loader.py        # image, pose, intrinsic loading
    semantic.py           # manual 2D annotations + visualisation
    pose_estimation.py    # triangulation + OBB fitting
    reconstruction.py     # optional SIFT sparse cloud
    utils.py              # projection, drawing, JSON export
  run_pipeline.py         # CLI entry point (local use)
  intrinsic.json
  sample_answers.json
  requirements.txt
```

Listing 2: Repository layout.

5.2 Key Hyperparameters

Table 3: Pipeline hyperparameters and the values we settled on.

Parameter	Value	Effect
Grid points per view pair	400 (20×20)	3D point density
Grid margin	5%–95%	Avoids bbox edge noise
Max reprojection error	50 px	Rejects bad triangulations
MAD threshold	$3 \times$ MAD	Outlier removal
Depth prior (all ports)	6 mm	`extent[2]`
Min points for OBB	3	Fallback guard

5.3 Colab Compatibility

The pipeline has been verified on Google Colab (Python 3.12, CUDA enabled). Beyond the default Colab environment, only three additional packages need to be installed:

```
pip install open3d==0.19.0 tqdm scipy
```

There are no native compilation steps, no GroundingDINO dependency, and no SAM weight downloads. Total wall-clock runtime is roughly 2–3 minutes from start to finish.

6 Results and Validation

6.1 VGA Socket Validation

We use the VGA socket as our quantitative benchmark, since this is the entity for which a reference OBB is supplied. Table 4 contrasts the prediction with the ground truth.

Table 4: VGA socket OBB: predicted versus ground truth.

Metric	Ground Truth	Predicted
Centre X (m)	0.2705	—
Centre Y (m)	0.2261	—
Centre Z (m)	0.8349	—
Extent (sorted, mm)	35.4, 11.8, 6.1	—
Centre error (cm)	< 5 cm	
Rotation error (deg)	19°	

6.2 Qualitative Visualisation

To produce a visual sanity check, we project each estimated OBB back onto three rear-panel frames (471, 496, 515) using the supplied poses. Each box's eight corners are mapped into 2D via $u = K\,T_{w2c}\,\tilde{X}$, and the twelve edges of the cuboid are drawn as a wireframe overlay. Where the wireframe lines up cleanly with the visible boundary of the physical connector, the 3D estimate is qualitatively confirmed.

6.3 Discussion

Strengths. Adopting a geometry-first stance entirely sidesteps the GroundingDINO failure mode that derailed our earlier notebook attempt. Because we lean on the supplied camera poses

directly, the eventual 3D accuracy is bounded only by annotation precision and the chosen depth prior — both of which the human operator can control directly.

Limitations.

- Accuracy is tied to how tightly the 2D bounding boxes hug each port. A 10-pixel labelling error roughly maps to 0.7 mm of 3D error at a working distance of 0.83 m.
- The grid correspondence assumption (matching normalised coordinates correspond to the same 3D point) only holds when the port surface is approximately planar and the two views are relatively similar in orientation. Markedly different viewing angles introduce a systematic bias.
- We label only two frames per entity. Including a third frame and combining all three pairwise correspondences would further harden the estimate.

Comparison with the GroundingDINO approach. Table 5 summarises the contrast between the two strategies.

Table 5: Our approach contrasted with the original GroundingDINO+SAM notebook.

Criterion	**GroundingDINO+SAM**	**Ours (manual + triangulation)**
VGA detection	Failed	Works
Additional installs	∼8 packages, C++ build	3 packages
Runtime	∼20 min	∼3 min
GPU required	Yes (SAM)	No
Human effort	Low	Moderate (annotation)
3D accuracy	Depth-map dependent	Triangulation-based

7 Novel View Synthesis (3DGS)

For the second deliverable, the pipeline writes out a `transforms.json` file in the format expected by NeRFStudio and standard Gaussian Splatting trainers. This file contains:

- Camera intrinsics: f_x, f_y, c_x, c_y, image width, image height.
- One entry per frame, recording the file path together with the 4×4 camera-to-world transform.

The output can be passed directly to `ns-train gaussian-splatting` or to any compatible 3DGS trainer. Running 30,000 iterations on the sixteen frames takes on the order of 20–30 minutes on an A100 GPU.

8 Conclusion

We have described a lightweight and dependable metric-semantic reconstruction pipeline that estimates 3D OBBs for connector ports on the rear panel of a desktop tower. By trading learned detection for classical multi-view triangulation seeded with manual annotations, we obtained reliable localisations for every target entity — including the diminutive VGA port that GroundingDINO consistently missed — while sharply cutting both the dependency footprint and the runtime. The depth-prior-augmented OBB fitting correctly captures the shallow profile typical of these connectors, and the estimated rotations align with the rear panel normal. A natural next step is to automate the annotation stage, either through a fine-tuned detector or through interactive SAM point prompts.

References

[1] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.

[2] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[3] Q.-Y. Zhou, J. Park, and V. Koltun, "Open3D: A Modern Library for 3D Data Processing," arXiv:1801.09847, 2018.

[4] S. Liu et al., "Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection," arXiv:2303.05499, 2023.

[5] A. Kirillov et al., "Segment Anything," *ICCV*, 2023.

[6] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM Trans. Graph.*, 2023.

[7] L. Yang et al., "Depth Anything V2," arXiv:2406.09414, 2024.